

Guía de Implementación: SQLite

Himnario 2.0 - Flutter Multiplataforma (Linux / Windows / Android)

Esta guía detalla cómo configurar, inicializar y estructurar la base de datos local embebida (SQLite) en Flutter, soportando entornos de escritorio (como tu entorno de desarrollo en Ubuntu) y dispositivos móviles.

1. Dependencias en `pubspec.yaml`

Para lograr la compatibilidad entre Android (móvil) y Ubuntu/Windows (Desktop), necesitas agregar estas dependencias a tu proyecto Flutter:

```
dependencies:  
  flutter:  
    sdk: flutter  
  sqflite: ^2.3.0      # Motor principal (Android/iOS)  
  sqflite_common_ffi: ^2.3.0 # Soporte nativo para Desktop (Linux/Windows)  
  path_provider: ^2.1.1  # Para encontrar la ruta de documentos del SO  
  path: ^1.8.3           # Utilidades de rutas de archivos
```

2. Inicialización Multiplataforma (Dart)

El siguiente código demuestra cómo inicializar la base de datos detectando el sistema operativo. En la arquitectura limpia, esto iría en `core/database/database\_helper.dart`.

```

import 'dart:io';
import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';
import 'package:path/path.dart';
import 'package:path_provider/path_provider.dart';

class DatabaseHelper {
  static Database? _database;

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('himnario_v2.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    // 1. Configurar FFI si estamos en Linux o Windows
    if (Platform.isWindows || Platform.isLinux) {
      sqfliteFfiInit();
      databaseFactory = databaseFactoryFfi;
    }

    // 2. Obtener ruta donde se guardará el archivo físico
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);

    // 3. Abrir la base de datos y crear tablas si no existe
    return await openDatabase(
      path,
      version: 1,
      onCreate: _createSchema,
    );
  }
}

```

3. Creación del Esquema Relacional (SQL)

La función `\_createSchema` ejecutará el código SQL para inicializar las tablas de tu Himnario, incluyendo el sistema de "Forks" (Arreglos musicales).

```

Future _createSchema(Database db, int version) async {
  // Tabla Principal de Himnos
  await db.execute('''
    CREATE TABLE Himno (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      titulo_principal TEXT NOT NULL,
      numero_oficial INTEGER,
      tipo INTEGER NOT NULL -- 1:Oficial, 2:Inspirada, 3:Convencion
    )
  ''');

  // Tabla de Usuarios (Para gestionar los "Forks")
  await db.execute('''
    CREATE TABLE Usuario (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      nombre TEXT NOT NULL,
      rol TEXT NOT NULL
    )
  ''');

  // Tabla de Estrofas Oficiales (Formato ChordPro)
  await db.execute('''
    CREATE TABLE Estrofa (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      himno_id INTEGER NOT NULL,
      tipo TEXT NOT NULL, -- Coro, Estrofa, Puente
      orden INTEGER NOT NULL,
      contenido TEXT NOT NULL, -- Ej: Señor mi [G]Dios...
      FOREIGN KEY (himno_id) REFERENCES Himno(id)
    )
  ''');

  // Tabla de Arreglos Personalizados (Forks)
  await db.execute('''
    CREATE TABLE Arreglo_Musical (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      himno_id INTEGER NOT NULL,
      usuario_id INTEGER NOT NULL,
      nombre_arreglo TEXT NOT NULL,
      tonalidad_base TEXT NOT NULL,
      FOREIGN KEY (himno_id) REFERENCES Himno(id),
      FOREIGN KEY (usuario_id) REFERENCES Usuario(id)
    )
  ''');

  // Estrofas modificadas por el usuario
  await db.execute('''
    CREATE TABLE Estrofa_Arreglo (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      arreglo_musical_id INTEGER NOT NULL,
      tipo TEXT NOT NULL,
  
```

```
orden INTEGER NOT NULL,  
contenido TEXT NOT NULL,  
FOREIGN KEY (arreglo_musical_id) REFERENCES Arreglo_Musical(id)  
)  
''');  
}  
}
```

Nota de Arquitectura sobre ChordPro: Recuerda que el campo contenido en las tablas de Estrofas almacena la letra Y los acordes juntos. No dividas los acordes en otra tabla. La extracción y transposición se hará en Dart (Frontend).

4. Herramientas para tu Entorno Ubuntu

DB Browser for SQLite

Como no usarás Docker Compose para levantar servicios de bases de datos locales, la mejor manera de ver e insertar datos de prueba en tu archivo .db generado es usando un cliente gráfico local.

Instalación en la terminal de Ubuntu:

```
sudo apt update  
sudo apt install sqlitebrowser
```

Simplemente abres el programa, buscas el archivo himnario_v2.db que Flutter generará en tu carpeta de usuario de Linux, y podrás ejecutar sentencias SQL, ver las relaciones de tus tablas y agregar himnos manualmente para probar.